

G
Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Centro de Rescate Animal: Gestión de Refugio y Adopciones

PONDERACIÓN: 25 pts

 **Tiempo estimado: 67 hrs**

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART	3
3. Resumen Ejecutivo	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema o necesidad a resolver	4
4.2 Alcance del proyecto	4
4.3 Recursos y herramientas a utilizar	4
4.4 Entregables	4
5. Material de apoyo	5
6. Metodología	5
7. Cronograma	6
8. Rúbrica de Calificación	6
8.1 Requisitos para optar a la calificación	6
8.2 Detalle de la Calificación	7
8.3 Comentarios Generales	8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Honestidad académica	El estudiante debe desarrollar su propia solución con arreglos y matrices, explicar su código en la revisión y evidenciar avance mediante commits progresivos.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia:

- Utiliza arreglos estáticos y matrices en Java para registrar animales, adoptantes, solicitudes, rescates, áreas y capacidad del refugio, aplicando recorridos, búsquedas, validaciones y actualización de datos sin depender de estructuras avanzadas no vistas en clase.
- Desarrolla una aplicación con interfaz gráfica construida por código en Java Swing, con ventanas, botones, tablas, campos de texto, mensajes de validación y navegación entre módulos, sin utilizar herramientas visuales de arrastrar y soltar.
- Aplica ciclos, condicionales, métodos y validaciones para controlar el flujo del programa, evitando registros duplicados, datos vacíos y operaciones inválidas.
- Implementa validaciones de datos, persistencia en archivos de texto y generación de reportes HTML para el seguimiento operativo del refugio.
- Utiliza Git/GitHub con commits semanales, mensajes semánticos y evidencia clara del avance real del proyecto.
- Documenta técnica y funcionalmente el sistema mediante manual de usuario, manual técnico, diagramas y bitácora de pruebas.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- Organización y planificación del trabajo individual.
- Comunicación técnica al defender decisiones de diseño y explicar el código desarrollado.
- Resolución de problemas ante errores de validación, búsqueda, actualización de datos y manejo de archivos.
- Atención al detalle en el menú gráfico, documentación, pruebas y control de versiones.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Desarrollar un sistema de interfaz gráfica en Java para administrar animales, adoptantes, solicitudes, rescates y espacios físicos de un refugio.	Cumplir los módulos funcionales solicitados, uso correcto de arreglos estáticos y matrices, reportes HTML y persistencia en archivos.	Aplicando Java con interfaz gráfica programada en Swing, archivos, métodos, validaciones, arreglos estáticos, matrices, ciclos y condicionales.	Para integrar los temas del enunciado inicial en un caso práctico verificable durante una defensa técnica.	Durante las 67 horas estimadas y antes de la fecha de entrega definida en el cronograma.

3. Resumen Ejecutivo

El proyecto consiste en desarrollar una aplicación con interfaz gráfica construida por código llamada Centro de Rescate Animal: Gestión de Refugio y Adopciones. La solución permitirá registrar animales rescatados, controlar sus estados clínicos y de adopción, administrar adoptantes, gestionar solicitudes y representar la distribución de espacios del refugio mediante arreglos estáticos y matrices. La interacción con el usuario deberá realizarse mediante ventanas programadas manualmente en Java Swing, evitando el uso de generadores visuales de formularios.

Para reducir la posibilidad de desarrollo automático sin comprensión, el proyecto exige defensa oral, modificación de código en vivo, bitácora de decisiones, commits incrementales y escenarios de prueba generados a partir del carné del estudiante.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Un centro de rescate animal recibe perros y gatos provenientes de reportes ciudadanos. Cada animal debe ser registrado, evaluado por veterinaria, ubicado en un espacio físico disponible y posteriormente asignado a un proceso de adopción. Actualmente, el control se realiza de forma manual, lo que provoca pérdidas de información, duplicidad de registros, confusión en solicitudes de adopción y poca visibilidad sobre la ocupación del refugio.

El estudiante deberá construir una aplicación gráfica que permita administrar el ciclo básico del refugio: ingreso del animal, historial básico, ubicación, solicitudes, adopciones, rescates urgentes, reportes y consulta de datos. La lógica deberá estar implementada mediante arreglos estáticos, matrices, ciclos, condicionales, métodos, validaciones, archivos y reportes HTML. La interfaz deberá programarse directamente con código Java Swing.

4.2 Alcance del proyecto

Alcance obligatorio:

- Módulo de autenticación local con usuario administrador y usuario auxiliar, cargados desde archivo.
- Módulo de animales rescatados: crear, buscar, editar estado, listar y eliminar lógicamente animales.
- Registro de animales mediante arreglos estáticos de objetos o arreglos paralelos, con búsqueda por código, nombre, especie y estado.
- Módulo de adoptantes administrado mediante arreglos estáticos, permitiendo registrar, buscar, editar y listar posibles adoptantes.
- Módulo de solicitudes de adopción usando arreglos estáticos, permitiendo registrar solicitudes, cambiar estado, atender solicitudes pendientes y consultar historial.
- Módulo de rescates urgentes usando arreglos estáticos, permitiendo registrar casos, asignar prioridad, atender casos y consultar reportes activos.
- Panel de ubicaciones del refugio basado en matrices: filas por área y columnas por jaula/espacio. Debe permitir consultar disponibilidad, asignar animales, liberar espacios y validar la capacidad máxima de cada zona.
- Interfaz gráfica principal para gestionar animales, adoptantes, solicitudes, rescates, ubicaciones del refugio, reportes y datos del estudiante, utilizando Java Swing programado manualmente.
- Persistencia en archivos de texto o binarios para recuperar datos al reiniciar la aplicación.
- Reportes HTML de animales, adopciones, ocupación del refugio y bitácora de acciones.
- La interfaz gráfica debe ser desarrollada únicamente mediante código Java Swing. No se permite utilizar diseñadores visuales, formularios generados automáticamente ni herramientas drag and drop.
- Manejo de errores y validaciones: campos vacíos, duplicados, valores inválidos, estados no permitidos y espacios ocupados.

Alcance opcional:

- Exportar reportes adicionales en PDF.
- Filtros por especie, estado, edad estimada o nivel de urgencia.
- Mejoras visuales en los reportes HTML generados.
- Generación de datos de prueba desde archivo CSV.
- Panel de estadísticas con porcentajes de ocupación y adopciones completadas.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Lenguaje	Java 17 o superior. Para el almacenamiento principal se deberán usar arreglos estáticos y matrices; no se permite usar Collections para resolver la lógica principal.
Obligatorio	Interfaz gráfica por código	Java Swing programado manualmente. Debe incluir ventanas, paneles, botones, campos de texto, tablas, mensajes de validación y navegación entre módulos. No se permite usar NetBeans GUI Builder, WindowBuilder, JavaFX Scene Builder ni herramientas de arrastrar y soltar.
Obligatorio	Control de versiones	Git y GitHub con commits incrementales y semánticos.
Obligatorio	Persistencia	Archivos .txt, .csv generados y leídos por la aplicación.
Obligatorio	Documentación	Manual técnico, manual de usuario, diagrama de clases, diagrama de flujo y bitácora de pruebas.
Obligatorio	IDE	NetBeans, IntelliJ IDEA, Eclipse o VS Code.
Obligatorio	Reportes	HTML con CSS básico generado desde Java.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Repositorio GitHub	Repositorio con formato IPC1_<SECCION>_2S2026_<CARNET>_P1. Debe incluir el código fuente, estructura de carpetas y evidencia de commits.
Código fuente	Proyecto Java completo, compilable y ejecutable. Debe contener clases o archivos separados para modelo, lógica del menú, persistencia y reportes.
Manual de Usuario	Documento PDF con capturas de pantalla y explicación del uso de cada módulo.
Manual Técnico	Documento PDF o Markdown con arquitectura, métodos principales, arreglos/matrices utilizados, validaciones, persistencia y reportes.
Diagramas	Diagrama de flujo del menú principal, diagrama de módulos y diagrama de la matriz de ubicaciones del refugio.
Reportes generados	Archivos HTML de animales, adopciones, ocupación y bitácora, con ejemplos reales generados por el sistema.
Hoja de calificación	Dos copias impresas el día de la revisión, según indicación del auxiliar.

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	Link	Descripción
Documentación oficial	Documentación oficial de Java	Consulta de sintaxis, clases, métodos, lectura/escritura de archivos y buenas prácticas básicas.
Manual	Apuntes del curso IPC1	Repaso de clases, objetos, archivos, ciclos, arreglos y modularización.
Ejemplo	Ejercicios de arreglos y matrices vistos en laboratorio	Base conceptual para manejar registros, búsquedas, recorridos, posiciones disponibles y matrices utilizando arreglos estáticos, sin ArrayList, HashMap, List, Queue ni estructuras similares.
Documentación oficial	Ejercicios de generación de reportes HTML	Referencia para construir archivos HTML desde Java usando cadenas, tablas y estilos básicos.

6. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

Tutor Academico (TA): Escribe una serie de pasos lógicos que el estudiante debe realizar para abordar el proyecto recuerda que ahora tu eres el experto y una guía para los estudiantes, cómo abordarías tu el proyecto.

Pasos a seguir:

1. Analizar el enunciado y definir las clases principales: Animal, Adoptante, Solicitud, Rescate, EspacioRefugio, Usuario y Bitacora.
2. Diseñar primero los arreglos estáticos y matrices que almacenarán la información del sistema, definiendo tamaños máximos, índices de control, búsquedas y validaciones antes de construir el menú gráfico.
3. Implementar persistencia básica para cargar y guardar animales, adoptantes, solicitudes y usuarios.
4. Construir la interfaz gráfica por módulos, evitando mezclar la lógica de presentación con la lógica de negocio. Cada ventana o panel debe llamar métodos específicos para registrar, buscar, editar, validar y generar reportes.
5. Implementar el módulo de ubicaciones mediante una matriz que permita mostrar en una tabla o panel gráfico los espacios ocupados y disponibles del refugio.

6. Generar reportes HTML y validar que los nombres de archivo incluyan fecha y hora.
7. Realizar pruebas con datos propios, documentar errores corregidos y registrar decisiones en la bitácora.
8. Preparar la defensa del proyecto explicando arreglos, matrices, flujo de datos, persistencia, reportes, validaciones y construcción manual de la interfaz gráfica.

Recomendaciones

- No iniciar por el diseño visual de las ventanas; primero validar el manejo de arreglos, matrices, búsquedas, registros y persistencia.
- Evitar clases gigantes. Separar modelo, servicios, ventanas/paneles Swing, persistencia, reportes y utilidades.
- Realizar commits por funcionalidad terminada, no solamente al final del proyecto.
- Preparar al menos tres escenarios de prueba: refugio vacío, refugio parcialmente ocupado y solicitudes acumuladas.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Entrega del enunciado	15/08/2026	12/09/2026
Avance 1: arreglos, matrices y persistencia	Semana 1	Semana 2
Avance 2: interfaz gráfica y ubicaciones	Semana 3	Semana 4
Avance 3: reportes, archivos y pruebas	Semana 5	Semana 6
Entrega final y revisión	Definir por cátedra	Definir por cátedra

8. Rúbrica de Calificación

La calificación total del proyecto será de 40 puntos. El estudiante deberá cumplir los requisitos mínimos para optar a calificación y demostrar dominio del código durante la revisión.

8.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Lenguaje	El proyecto está desarrollado en Java y es ejecutable.	
Interfaz gráfica por código	La aplicación utiliza ventanas programadas manualmente en Java Swing.	
Arreglos y matrices	Utiliza arreglos estáticos y matrices para almacenar y procesar los datos de los módulos principales. No utiliza ArrayList, LinkedList, HashMap, List, Queue, Stack, Vector ni estructuras equivalentes del Java Collections Framework.	
Repositorio	El repositorio cumple el formato solicitado y contiene commits progresivos.	
Defensa	El estudiante puede explicar y modificar su código durante la revisión.	
Entregables	Incluye manuales, diagramas, reportes, bitácora y hoja de calificación.	

8.2 Detalle de la Calificación

No.	Criterio de evaluación	Punteo máximo	Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo obtenido
1	Funcionalidad del sistema	12	Implementa animales, adoptantes, solicitudes, rescates, ubicaciones, reportes e interfaz gráfica de forma integrada.	Módulos incompletos, errores frecuentes o flujo de datos inconsistente.	
1.1	Gestión de animales y adoptantes	4	Permite crear, buscar, editar estado y consultar datos sin duplicados ni errores de validación.	Registros incompletos, búsquedas deficientes o validaciones insuficientes.	
1.2	Solicitudes, rescates y adopciones	3	Administra solicitudes y rescates con arreglos estáticos, estados y prioridades correctamente validadas.	No administra correctamente estados, prioridades o presenta pérdida de datos.	
1.3	Matriz de ubicaciones	3	Representa espacios con matrices y permite consultar, asignar y liberar ubicaciones.	La matriz no controla disponibilidad, capacidad o asignación de ubicaciones de forma correcta.	

1.4	Reportes y persistencia	2	Guarda/carga información y genera reportes HTML correctos.	No persiste datos o los reportes son incompletos.	
2	Conocimientos técnicos	16	Demuestra dominio de arreglos, matrices, ciclos, condicionales, métodos, archivos, reportes HTML y construcción manual de interfaces Swing.	No puede explicar decisiones o el código evidencia dependencia de soluciones copiadas/generadas sin comprensión.	
2.1	Arreglos y matrices	5	Implementa arreglos estáticos y matrices, con métodos claros para registrar, buscar, editar, eliminar lógicamente, recorrer y validar capacidad. No utiliza estructuras dinámicas o colecciones de Java.	Usa ArrayList, HashMap, List, Queue, Stack, Vector, colecciones de Java, estructuras no permitidas o no logra manejar correctamente los arreglos/matrices.	
2.2	Interfaz gráfica por código	3	Ventanas, botones, tablas y navegación programadas manualmente, con validaciones y retorno correcto a las pantallas principales.	Interfaz incompleta, confusa, construida con herramientas visuales no permitidas o sin validaciones básicas.	

2.3	Archivos y reportes HTML	4	Lectura/escritura de archivos funcional y reportes HTML con formato solicitado.	Archivos no persistentes, reportes incompletos o formato incorrecto.	
2.4	Validaciones y manejo de errores	2	Controla entradas inválidas, duplicados y estados no permitidos.	Se cae ante entradas básicas o datos inconsistentes.	
2.5	Cambio de código en vivo	2	Realiza la modificación solicitada y explica su impacto.	No logra modificar el código o no comprende el flujo.	
3	Documentación y entrega	8	Entrega completa, ordenada y coherente con el sistema implementado.	Faltan manuales, diagramas, reportes o existe desorden en el repositorio.	
3.1	Manual técnico y diagramas	3	Describe métodos, arreglos, matrices, archivos, reportes y diagramas solicitados.	Documentación superficial o no coincide con el código.	
3.2	Manual de usuario y capturas	2	Explica el uso de cada módulo con capturas reales.	Manual incompleto o sin evidencias.	
3.3	GitHub y commits	2	Repositorio con estructura correcta, commits semanales y colaborador agregado.	Repositorio mal nombrado, sin historial suficiente o sin colaborador.	
3.4	Bitácora anti-IA	1	Incluye decisiones, pruebas, problemas y soluciones propias.	Bitácora ausente o genérica.	

4	Habilidades de defensa	4	Expone con claridad, responde preguntas y demuestra autoría del desarrollo.	Respuestas vagas, contradicciones o incapacidad de explicar el código.	
	Total	40			

8.3 Penalizaciones y Comentarios Generales

Criterio	Descripción	Penalización
Entrega fuera de tiempo	El repositorio o entregables se modifican después de la hora límite sin autorización.	-100%
Uso de librerías prohibidas	Uso de ArrayList, LinkedList, HashMap, List, Queue, Stack, Vector u otras estructuras del Java Collections Framework para resolver almacenamiento, búsquedas, recorridos, reportes o lógica principal del proyecto.	Hasta -75%
Código generado o copiado	Uso mayoritario de IA, copia entre compañeros, proyectos de semestres anteriores o código externo no explicado.	-100%

No defensa del código	El estudiante no puede explicar arreglos, matrices, validaciones, archivos, flujo del programa o construcción manual de la interfaz gráfica.	Hasta -100%
Commits insuficientes	No cumple con el mínimo de commits semanales o todos los commits están concentrados al final.	-20%
Repositorio mal organizado	Archivos fuera de lugar, falta de README o proyecto no compilable desde la estructura entregada.	Hasta -25%
Documentación inconsistente	Manuales, diagramas o reportes no coinciden con el sistema presentado.	Hasta -30%
